

Submission Worksheet

Submission Data

Course: IT114-004-S2026

Assignment: IT114 Milestone 2 - Generic Project Submission

Student: Stefan I. (si329)

Status: Submitted | **Worksheet Progress:** 99%

Potential Grade: 9.95/10.00 (99.50%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/16/2026 10:11:31 PM

Updated: 4/21/2026 12:40:37 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-004-S2026/it114-milestone-2-generic-project-submission/grading/si329>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-004-S2026/it114-milestone-2-generic-project-submission/view/si329>

Instructions

1. Refer to the Milestone 2 requirements for your approved project proposal and complete the required features before filling this out.
 - Project proposal docs:
 - [Rock Paper Scissors](#)
 - [Hangman](#)
 - [Trivia](#)
 - [Basic Battleship](#)
 - [Drawing](#)
 - [Go Fish](#)
 - This worksheet includes project hints, but your full project proposal document is the source of truth for project-specific requirements.
 2. Ensure all code snippets include your UCID, date, and a brief description of what the code does.
 3. Switch to the Milestone2 branch.
 1. `git checkout Milestone2`
 2. `git pull origin Milestone2`
 4. Fill out the worksheet below as you test and demo with 3+ clients in the same session.
 5. Once finished, click "Submit and Export".
 6. Add the generated PDF somewhere inside your repository and move it to GitHub.
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone2`
 4. On GitHub merge the pull request from Milestone2 to main
 7. Upload the same PDF to Canvas.
 8. Sync local.
 1. `git checkout main`
 2. `git pull origin main`
- Review the details and validation criteria for each task.

Section #1: (1 pt.) Project Setup

Progress: 100%

⇒ Task #1 (0.25 pts.) - Choose Your Project Type

Progress: 100%

Details:

Select and confirm your Milestone 2 project from the options below. Briefly explain what draws you to this particular project.

Project options:

- Rock Paper Scissors
- Hangman
- Trivia
- Basic Battleship
- Drawing
- Go Fish

Your Response:

I chose Rock Paper Scissors because it seemed to be the most simple out of the options as it didn't require anything on the end of GameServer to be changed beyond simply taking in the inputs and then resolving points.

This is an edit from later. It was not that simple.



Saved: 4/20/2026 10:27:25 PM

≡ Task #2 (0.75 pts.) - Show Payload classes and subclasses

Progress: 100%

Details:

- Include the shared Payload types used for your Milestone 2 flow.
 - No need to include the base `Payload` class since it shouldn't have changed for Milestone 2 (some projects can work with just this class for core actions).
 - If your project uses `BoolPayload`, include it.
- Project hints:
 - Drawing / Battleship: show your coordinate and dimension related payloads
 - Trivia: show `QAPayload` or equivalent question payload
 - Go Fish: show target / hand / card-related payloads
 - RPS / Hangman: `PointsPayload`, `BoolPayload` (if used), plus project-specific sync payloads may be enough if you kept the data simple

Part 1:

Progress: 100%

Details:

- (code) Show your primary action payload(s) and `PointsPayload`.
- (code) Each payload should have an overridden `toString()` that shows its internal data.
- (runtime) Include at least one log of a primary action payload being sent or received, and one `PointsPayload` update (log in `processPayload()` if needed).

```
package Project.Common;

You, 4 days ago | 1 author (You)
public class BoolPayload extends Payload {
    private boolean value;
    // You, 4 days ago + Added baseline files for mltcstone2
    /**
     * Return the boolean value carried by this payload
     */
    public boolean getValue() {
        return value;
    }
    /**
     * Set the boolean value to the boolean value to set
     */
    public void setValue(boolean value) {
        this.value = value;
    }
    @Override
    public String toString() {
        return super.toString() + String.format(" Value: %s", getValue());
    }
}
```

Shows boolpayload.java, not edited from the baseline

```
package Project.Common;

You, 4 days ago | 1 author (You)
public class PointsPayload extends Payload {
    private int points;
    public PointsPayload() {
        // Some payloads will always/generally have the same type
        // so we can set it in the constructor to reduce repeated code when creating the
        // payload
        setPayloadType(PayloadType.POINTS);
    }
    public PointsPayload(int points) {
        super(); // call default constructor to set payload type
        this.points = points;
    }
    public int getPoints() {
        return points;
    }
    public void setPoints(int points) {
        this.points = points;
    }
    @Override
    public String toString() {
        return super.toString() + String.format(" Points: %d", points);
    }
}
```

Shows points payload

```
package Project.Common;

public enum PayloadType {
    CLIENT_CONNECT, // client requesting to connect to server (passing of initialization data)
    CLIENT_ID, // server sending client id
    DISCONNECT, // distinct disconnect action
    REVERSE,
    MESSAGE, // sender and message
    SERVER_JOIN, // server notifying recipient of a new client joining
    // (includes new client's id and name)
    SERVER_LEAVE, // server notifying recipient of a client leaving
    // (includes leaving client's id and name)
    SERVER_SYNC, // server notifying recipient of existing client
    // (includes existing client's id and name) (silently)
    READY, // client signals ready check participation
    TURN, // client submits a turn action
    GAME_PHASE_SYNC, // server syncs current game phase to clients
    PLAYER_READY_STATUS, // server syncs whether a user is ready
    PLAYER_TURN_STATUS, // server syncs whether a user already took a turn
    POINTS, // server syncs a user's points
    CHOICE, // Replaces the guess functionality of the old code
    PLAYER_ELIMINATED, // You, 2 hours ago + Uncommitted changes
}
```

Included just to make sure I have everything here

```
> [ClientId: 1] Sending to client: Payload[MESSAGE] Client Id [0] Message: [Server: [Game] Round 1 started. Use /choice <r/p/s> to make your pick. You have 30s.]
04/20/2026 23:15:44 [Project.Server.ServerThread] [INFO]:
> [ClientId: 2] Sending to client: Payload[MESSAGE] Client Id [0] Message: [Server: [Game] Round 1 started. Use /choice <r/p/s> to make your pick. You have 30s.]
04/20/2026 23:15:44 [Project.Server.GameServer] [INFO]:
> [GameServer] onRoundStart() end
04/20/2026 23:15:45 [Project.Server.GameServer] [INFO]:
> [GameServer] Round timer: 29
04/20/2026 23:15:45 [Project.Server.GameServer] [INFO]:
```

```
04/20/2026 23:15:4 [Project.Server.ServerThread] (INFO):  
> [ClientId: 2] Received from client: Payload[CHOICE] Client Id [0] Message: {rock}
```

Shows primary action payload being sent/received

```
04/20/2026 23:11:46 [Project.Server.ServerThread] (INFO):  
> [ClientId: 1] Sending to client: Payload[POINTS] Client Id [2] Message: {null} Points [=1]
```

Shows runtime points payload



Saved: 4/20/2026 11:25:06 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown and why it exists instead of reusing the base `Payload`.

Your Response:

Each payload allows us to encapsulate different data. We extend off of payload and then use our specific desired datatypes, in the example of Points payload, it's an integer, in the case of a bool, it's a boolean value. This helps prevent conflict between datatypes, as if something is a `PointPayload`, it logically cannot be anything else. The base `Payload` is just for a string, which means we would have to parse this into something useful every time. Instead of this, we can just go directly to using exclusively integers and booleans



Saved: 4/20/2026 11:25:06 PM

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

Task #1 (0.67 pts.) - GameRoom Client Add/Remove

Progress: 100%

Details:

- Baseline method naming convention: `processClientCommand(...)` (client) → `process*()` (inbound routing) → `handle*()` (authoritative logic) → `broadcast / unicast / multicast (sync)` → `send*()` (outbound). Lifecycle hooks:

- Show how joiners receive enough current state to understand the session and how leaving users are cleaned up.
- Concisely note what gets synchronized to a joiner during an active session.
- Show that join sync is phase-aware (for example inactive/ready vs active session behavior) and uses unicast state replay where appropriate.
- Mid-session joiners are non-participants for the current active session; state and demonstrate that behavior explicitly.
- Project hints:
 - Drawing / Battleship: board dimensions and current known board state
 - Hangman: blanks, strikes, current player, and turn status
 - Trivia: current question, options, round state, and answer-lock status
 - Go Fish: current player, hand counts, deck status, and turn status
 - RPS: current phase, points, round state, acted status, and elimination

Progress: 100%

- **(code)** Show your join hook (`onPlayerJoined` / `onClientAdded`) and leave hook (`onPlayerLeft` / `onClientRemoved`) code using your baseline naming.
- **(runtime)** Include runtime evidence showing a join and a leave path such as the `/users` command or some other utility added to inspect local game state.
- **(runtime)** Include runtime evidence of a mid-session join where the joiner receives a unicast state snapshot before the next action.
- **(runtime)** hints for this task:
 - RPS: show join sync for phase, acted state, elimination, and points
 - Hangman / Trivia: show join sync for current prompt state (blanks or question/options)
 - Battleship / Drawing: show join sync for board dimensions/state
 - Go Fish: show join sync for current player and hand-count/deck status

Shows the code for on player join/leaving

```
private void unicastGameStateToJoiner(ServerThread joiner) {
    if (joiner == null) {
        return;
    }
}
```

```

        unicastCurrentPhase(joiner);
        List<ServerThread> snapshot = new ArrayList<>(getActivePlayers());
        for (ServerThread player : snapshot) {
            if (player.getClientId() == joiner.getClientId()) {
                continue;
            }
            unicastReadyStatus(joiner, player.getClientId(), player.isReady());
            unicastTurnStatus(joiner, player.getClientId(), player.isTurnTaken());
            unicastPlayerPoints(joiner, player.getClientId(), player.getPoints());
        }
    }
}

```

Shows unicasting for information that occurs

```

04/20/2026 23:31:07 [Project.Server.ServerThread] (INFO):
> [ClientId: 2] Processing disconnect payload
04/20/2026 23:31:07 [Project.Server.ServerThread] (INFO):
> [ClientId: 2] Sending to client: Payload[DISCONNECT] Client Id [0] Message: [n
ull]
04/20/2026 23:31:07 [Project.Server.ServerThread] (INFO):
> [ClientId: 2] Thread being disconnected by server
04/20/2026 23:31:07 [Project.Server.BaseServer] (INFO):
> Client Marisa#2 disconnected.
04/20/2026 23:31:07 [Project.Server.BaseGameServer] (INFO):
> [GameServer] Client removed: Marisa#2
04/20/2026 23:31:07 [Project.Server.BaseGameServer] (INFO):
> [GameServer] Client removed: Marisa#2
04/20/2026 23:31:07 [Project.Server.ServerThread] (INFO):
> [ClientId: 2] Exited thread loop. Cleaning up connection

```

I ran /disconnect and it disconnected



Saved: 4/21/2026 12:21:15 AM

⇒ Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in your join and leave hooks.
- Mention any empty-session, turn-holder-left, or stale-state cleanup logic if applicable.

Your Response:

Join hook: When a player joins via ready check, the server unicasts the current game session to them if a session is already in progress. Leave hook: When leave hook is called, the server checks to see if the player count falls below the threshold of two and then simply concludes the game if it does.



Saved: 4/21/2026 12:21:15 AM

≡ Task #2 (0.67 pts.) - GameRoom Session Start

Progress: 100%

Details:

- Show how authoritative session state is initialized and how the first round or phase begins.
- Project hints:
 - RPS: initialize remaining players, scores, and choice state
 - Hangman / Drawing: load word list data and reset player/session state
 - Trivia: load question data and reset player/session state

- `mivia`. load question data and reset player/session state
- Go Fish: build and shuffle deck, set turn order, deal starting hands
- Battleship: initialize player state, board dimensions, empty boards, and placement phase

Part 1:

Progress: 100%

Details:

- **(code)** Show the snippet of `onSessionStart()`.
- **(runtime)** Include runtime evidence showing session start initialization output.

```
protected synchronized void onSessionStart() {
    LoggerUtil.INSTANCE.info(message: "[GameServer] onSessionStart() start");
    resetReadyTimer();
    roundNumber = 0;
    broadcastGameMessage(message: "Session started.");
    LoggerUtil.INSTANCE.info(message: "[GameServer] onSessionStart() end");
    onRoundStart();
}
```

Shows on session start

```
> Server: [Game] Ready check started. Session begins in 30s if enough players are ready.
/ready
04/20/2026 23:37:43 [Project.Client.Client] (INFO):
> Processing command: READY
04/20/2026 23:37:43 [Project.Client.Client] (INFO):
> [Game] Joined as active player. Waiting room status: 2/2 ready.
04/20/2026 23:37:43 [Project.Client.Client] (INFO):
> Server: [Game] Marisa#3 joined active players.
04/20/2026 23:37:43 [Project.Client.Client] (INFO):
> Server: [Game] Active players: 2
04/20/2026 23:37:43 [Project.Client.Client] (INFO):
> Server: [Game] Marisa#3 is ready. (2 active)
04/20/2026 23:38:06 [Project.Client.Client] (INFO):
> Server: [Game] Session started.
04/20/2026 23:38:06 [Project.Client.Client] (INFO):
> [Game] Phase: IN PROGRESS
04/20/2026 23:38:06 [Project.Client.Client] (INFO):
> Server: [Game] Round 1 started. Use /choice <r/p/s> to make your pick. You have 30s.
```

Shows runtime start

 Saved: 4/21/2026 12:22:47 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the initial state created here and which lifecycle method happens next.

Your Response:

`onSessionStart` starts the session by resetting the round number to 0. It then calls the lifecycle method for starting the round, which is `onMethodStart`

Progress: 100%

- Show how your project prepares the active round, phase, or prompt that players interact with.
- Project hints:
 - RPS: reset choices, set choosing phase, start round timer
 - Trivia: choose question, clear lock-ins, broadcast choices, start round timer
 - Drawing: choose drawer and word, unicast the word, broadcast blanks, start round timer
 - Hangman: choose or reset word data, blanks, and strikes before turn flow
 - Go Fish: reset round state and trigger turn flow
 - Battleship: either setup placement flow or transition to attack flow depending on phase

Progress: 100%

- **(code)** Show the snippet of `onRoundStart()` or your equivalent phase-start method.
- **(runtime)** Include runtime evidence showing the round or phase start output.

```
protected synchronized void onRoundStart() {
    LoggerUtil.INSTANCE.info(message: "[GameServer] onRoundStart() start");
    resetRoundTimer();
    phase = Phase.IN_PROGRESS;
    broadcastCurrentPhase();
    for (ServerThread player : getActivePlayers()) {
        if (!player.isEliminated()) {
            player.setChoice(choice: null);
            player.setTurnTaken(turnTaken: false);
            broadcastTurnStatus(player.getClientId(), hasTakenTurn: false);
        }
    }
    roundNumber++;
    startRoundTimer();
    broadcastGameMessage("Round " + roundNumber + " started. Use /choice <r/p/s> to make your pick. You have " + ROUND_SECONDS + "s.");
    LoggerUtil.INSTANCE.info(message: "[GameServer] onRoundStart() end");
}
```

Shows on round start

```
04/20/2026 23:37:43 [Project.Client.Client] (INFO):  
[Game] Players joined active players.  
04/20/2026 23:37:43 [Project.Client.Client] (INFO):  
[Game] Round 2  
04/20/2026 23:37:43 [Project.Client.Client] (INFO):  
[Game] Round 2 is ready (2 active)  
04/20/2026 23:38:08 [Project.Client.Client] (INFO):  
[Game] Game was started.  
04/20/2026 23:38:08 [Project.Client.Client] (INFO):  
[Game] IN PROGRESS  
04/20/2026 23:38:08 [Project.Client.Client] (INFO):  
[Game] Round 1 started. Use /choice <r/p/> to make your pick. You have 30s.  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Choice 1  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Round ended. Evaluating choices.  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Choice 1  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Active was eliminated for not making a choice  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Player 1 was eliminated.  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Round ended for not making a choice  
04/20/2026 23:38:36 [Project.Client.Client] (INFO):  
[Game] Game was ended in a tie
```

Shows the runtime for the start of the round



Saved: 4/21/2026 12:25:18 AM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here and what data is synchronized to clients.

Your Response:

onRoundStart cancels any existing round timer, and starts a fresh one. It then sets phase to IN_PROGRESS, and broadcasts this to all clients using GAME_PHASE_SYNC. It loops through all active players, skips eliminated ones, sets choice and turntaken to false for eligible players, then broadcasts each players turn. Finally it increments round number



Saved: 4/21/2026 12:25:18 AM

≡ Task #4 (0.67 pts.) - Turn Control or Action Window Handling

Progress: 100%

Details:

- Show how your project controls when players are allowed to act.
- If your project is turn-based, show `onTurnStart()` and `onTurnEnd()`.
- If your project is simultaneous, show the logic that tracks who has acted or locked in and how the round can end early before the timer expires.
- Include the phase gate(s) and role/turn gate(s) that block invalid actions.
- Project hints:
 - Turn-based: Hangman, Go Fish, Battleship attack phase
 - Simultaneous: RPS, Trivia, Drawing, Battleship placement phase

🖼 Part 1:

Progress: 100%

Details:

- **(code)** Show the relevant turn-control or action-window snippet(s) (for example `onTurnStart()` / `onTurnEnd()` or equivalent).
- **(runtime)** Include runtime evidence showing turn ownership or action-window state changes.
- **(runtime)** hints for this task:
 - Turn-based (Hangman, Go Fish, Battleship attack): show current-player changes and timer expiry/advance

tracking and early-end condition handling



shows on turn start and on turn end method



shows handlechoice method



Runtime showing choice lockin

Saved: 4/21/2026 12:27:49 AM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain how the project knows a player can act, cannot act, or has finished acting for that window.

Your Response:

RPS is simultaneous: therefore all non-eliminated players can act as long as they act in the 30 second window. There are four checks facilitated by `handleChoice`, `requireParticipating`,

```

> [Game]: Phase: EVALUATION
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> Server: [Game] Round ended. Evaluating choices
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> Alice#1 now has 1 points
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> [Game] Round ended. All finished
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> Server: [Game] Alice#1 chose rock while Marina#3 chose scissors; Alice#1 wins!
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> Server: [Game] Alice#1 wins the session!
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> Server: [Game] Final Scoreboard:
1. Alice#1 - 1 points
2. Marina#3 - 0 points
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
> All users' ready status reset

```

```
04/20/2026 23:43:25 [Project.Client.Client] (INFO):  
> [Game] Phase: INACTIVE  
04/20/2026 23:43:25 [Project.Client.Client] (INFO):  
> Server: [Game] Session ended. Type /ready to join the next session.
```

Shows the runtime evidence of round evaluation



Saved: 4/21/2026 12:31:16 AM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the cleanup, scoring, elimination, and next-step logic that occurs here.

Your Response:

(phase == Phase.EVALUATION prevents the method from running twice. resetRoundTimer resets the timer (cleanup phase), !isTurnTaken eliminates anyone who didn't make a choice, resolveRPS (helper method) handles the logic



Saved: 4/21/2026 12:31:16 AM

≡ Task #6 (0.67 pts.) - GameRoom Session End

Progress: 100%

Details:

- Show the final win or tie logic, scoreboard output, and reset behavior for a new ready-check.
- Include cleanup of session-only state so no stale turn/phase/action data leaks into the next session.
- Project hints:
 - RPS / Battleship: last-player-standing or tie conditions
 - Hangman / Trivia / Drawing: fixed round-count ending
 - Go Fish: deck-empty or points threshold ending

🖼 Part 1:

Progress: 100%

Details:

- **(code)** Show the snippet of `onSessionEnd()`.
- **(runtime)** Include runtime evidence showing final scoreboard or end-of-session reset output.
- **(runtime)** Include evidence that a new ready-check/session starts from clean state (no stale in-progress data).

[illegible]

Shows onSessionEnd()

```
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
[Game] Marisa#3 has been eliminated
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
Server: [Game] Alice#1 chose rock while Marisa#3 chose scissors; Alice#1 wins!
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
Server: [Game] Alice#1 wins the session!
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
Server: [Game] Final Scoreboard:
1. Alice#1 - 1 points
1. Marisa#3 - 0 points
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
All users' ready status reset
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
[Game] Phase: INACTIVE
04/20/2026 23:43:25 [Project.Client.Client] (INFO):
Server: [Game] Session ended. Type /ready to join the next session.
```

Shows the final scoreboard as well as a clean slate



Saved: 4/21/2026 12:33:27 AM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the end condition, reset behavior, and what remains connected after the session ends.

Your Response:

onSessionEnd is triggered when remaining ≤ 1 onRoundEnd, either one player survived all the battles or all players were eliminated in a tie. The remaining non-eliminated player list is checked, size 1 means a winner is announced by name. A scoreboard is broadcasted, and then there is a resetGameState call which resets all of the players at the end of the round. Clients remain connected at the end of the round.



Saved: 4/21/2026 12:33:27 AM

Section #3: (4 pts.) Game State And Action Flow

Progress: 100%

:= Task #1 (1 pt) - Shared game state reference(s)

- Show the main server-authoritative game state and the related client-side reference or cached state.
- Project hints:
 - Drawing / Battleship: board or grid reference
 - Hangman: current word, blanks, strikes, guessed-state data
 - Trivia: current question, answer options, and lock-in tracking
 - RPS: remaining-player state, choices, elimination data
 - Go Fish: deck, per-player hand, and hand-count state

Progress: 100%

- **(code)** Show the client-side state reference code.
- **(code)** Show the server-side state reference code.
- **(runtime)** Show runtime output or another focused debug view that proves the state exists and updates.
- **(runtime)** hints for this task:
 - Battleship / Drawing: include board snapshots before and after an action
 - Hangman: include blanks/strikes snapshots across actions
 - Trivia: include question/lock-in state snapshots
 - RPS: include choice/remaining-player state snapshots
 - Go Fish: include hand/deck/hand-count state snapshots

```
case USERS: // client-side command
    StringBuilder sb = new StringBuilder();
    sb.append("Known clients:\n");
    knownUsers.values().forEach(c -> sb.append(String.format("%s%s Ready:%s Turn:%s Points:%s Choice:%s\n",
        c.getDisplayName(),
        c.getClientId() == myUser.getClientId() ? " (you)" : "",
        c.isReady() ? "[x]" : "[ ]",
        c.isTurnTaken() ? "[x]" : "[ ]",
        c.getPoints(),
        c.getGuess() == 0 ? "[?]" : c.getGuess())));
    // You, 4 hours ago • Changed ValidationUtils.
    c.getChoice() == null ? "[?]" : c.getChoice());
    LoggerUtil.INSTANCE.info(TextFX.colorize(sb.toString().trim(), Color.CYAN));
    return true;
}
```

Shows CASE users in on the client side

```

1  #!/usr/bin/perl -w
2
3  use strict;
4  use warnings;
5
6  my $VERSION = "1.0.0";
7  my $AUTHOR = "John Doe";
8  my $COPYRIGHT = "Copyright 2023 John Doe";
9
10 my $SYNOPSIS = "Usage: perl script.pl [options]";
11 my $DESCRIPTION = "This script is a simple example of a Perl script. It takes a single argument, a file name, and prints the contents of the file to the standard output.";
12 my $USAGE = "perl script.pl [file]";
13
14 my $help = "
15     -h, --help            Display this help message
16     -v, --version         Display the version number
17     -f, --file FILE       Specify the file to be processed
18     -o, --output FILE     Specify the output file
19     -s, --silent           Suppress the output
20     -q, --quiet           Suppress the output
21     -d, --debug           Enable debugging
22     -D, --dry-run         Dry run mode
23     -C, --config FILE     Specify the configuration file
24     -E, --environment     Specify the environment
25     -M, --module MODULE    Specify the module
26     -I, --include INCLUDE  Specify the include path
27     -L, --library LIBRARY  Specify the library
28     -P, --perl PERL        Specify the Perl interpreter
29     -S, --switch SWITCH    Specify the switch
30     -T, --test            Test mode
31     -X, --xargs            Use xargs
32     -Y, --yargs            Use yargs
33     -Z, --zargs            Use zargs
34     -A, --args            Use args
35     -B, --batch            Batch mode
36     -C, --compact          Compact mode
37     -D, --debug            Debug mode
38     -E, --error            Error mode
39     -F, --fast             Fast mode
40     -G, --gdb              GDB mode
41     -H, --help            Help mode
42     -I, --include          Include mode
43     -L, --library          Library mode
44     -M, --module           Module mode
45     -N, --no              No mode
46     -O, --output           Output mode
47     -P, --perl            Perl mode
48     -Q, --quiet           Quiet mode
49     -R, --recursive        Recursive mode
50     -S, --switch           Switch mode
51     -T, --test            Test mode
52     -U, --unbuffered        Unbuffered mode
53     -V, --version          Version mode
54     -W, --warning          Warning mode
55     -X, --xargs            Xargs mode
56     -Y, --yargs            Yargs mode
57     -Z, --zargs            Zargs mode
58     -A, --args            Args mode
59     -B, --batch            Batch mode
60     -C, --compact          Compact mode
61     -D, --debug            Debug mode
62     -E, --error            Error mode
63     -F, --fast             Fast mode
64     -G, --gdb              GDB mode
65     -H, --help            Help mode
66     -I, --include          Include mode
67     -L, --library          Library mode
68     -M, --module           Module mode
69     -N, --no              No mode
70     -O, --output           Output mode
71     -P, --perl            Perl mode
72     -Q, --quiet           Quiet mode
73     -R, --recursive        Recursive mode
74     -S, --switch           Switch mode
75     -T, --test            Test mode
76     -U, --unbuffered        Unbuffered mode
77     -V, --version          Version mode
78     -W, --warning          Warning mode
79     -X, --xargs            Xargs mode
80     -Y, --yargs            Yargs mode
81     -Z, --zargs            Zargs mode
82     -A, --args            Args mode
83     -B, --batch            Batch mode
84     -C, --compact          Compact mode
85     -D, --debug            Debug mode
86     -E, --error            Error mode
87     -F, --fast             Fast mode
88     -G, --gdb              GDB mode
89     -H, --help            Help mode
90     -I, --include          Include mode
91     -L, --library          Library mode
92     -M, --module           Module mode
93     -N, --no              No mode
94     -O, --output           Output mode
95     -P, --perl            Perl mode
96     -Q, --quiet           Quiet mode
97     -R, --recursive        Recursive mode
98     -S, --switch           Switch mode
99     -T, --test            Test mode
100    -U, --unbuffered        Unbuffered mode
101    -V, --version          Version mode
102    -W, --warning          Warning mode
103    -X, --xargs            Xargs mode
104    -Y, --yargs            Yargs mode
105    -Z, --zargs            Zargs mode
106    -A, --args            Args mode
107    -B, --batch            Batch mode
108    -C, --compact          Compact mode
109    -D, --debug            Debug mode
110    -E, --error            Error mode
111    -F, --fast             Fast mode
112    -G, --gdb              GDB mode
113    -H, --help            Help mode
114    -I, --include          Include mode
115    -L, --library          Library mode
116    -M, --module           Module mode
117    -N, --no              No mode
118    -O, --output           Output mode
119    -P, --perl            Perl mode
120    -Q, --quiet           Quiet mode
121    -R, --recursive        Recursive mode
122    -S, --switch           Switch mode
123    -T, --test            Test mode
124    -U, --unbuffered        Unbuffered mode
125    -V, --version          Version mode
126    -W, --warning          Warning mode
127    -X, --xargs            Xargs mode
128    -Y, --yargs            Yargs mode
129    -Z, --zargs            Zargs mode
130    -A, --args            Args mode
131    -B, --batch            Batch mode
132    -C, --compact          Compact mode
133    -D, --debug            Debug mode
134    -E, --error            Error mode
135    -F, --fast             Fast mode
136    -G, --gdb              GDB mode
137    -H, --help            Help mode
138    -I, --include          Include mode
139    -L, --library          Library mode
140    -M, --module           Module mode
141    -N, --no              No mode
142    -O, --output           Output mode
143    -P, --perl            Perl mode
144    -Q, --quiet           Quiet mode
145    -R, --recursive        Recursive mode
146    -S, --switch           Switch mode
147    -T, --test            Test mode
148    -U, --unbuffered        Unbuffered mode
149    -V, --version          Version mode
150    -W, --warning          Warning mode
151    -X, --xargs            Xargs mode
152    -Y, --yargs            Yargs mode
153    -Z, --zargs            Zargs mode
154    -A, --args            Args mode
155    -B, --batch            Batch mode
156    -C, --compact          Compact mode
157    -D, --debug            Debug mode
158    -E, --error            Error mode
159    -F, --fast             Fast mode
160    -G, --gdb              GDB mode
161    -H, --help            Help mode
162    -I, --include          Include mode
163    -L, --library          Library mode
164    -M, --module           Module mode
165    -N, --no              No mode
166    -O, --output           Output mode
167    -P, --perl            Perl mode
168    -Q, --quiet           Quiet mode
169    -R, --recursive        Recursive mode
170    -S, --switch           Switch mode
171    -T, --test            Test mode
172    -U, --unbuffered        Unbuffered mode
173    -V, --version          Version mode
174    -W, --warning          Warning mode
175    -X, --xargs            Xargs mode
176    -Y, --yargs            Yargs mode
177    -Z, --zargs            Zargs mode
178    -A, --args            Args mode
179    -B, --batch            Batch mode
180    -C, --compact          Compact mode
181    -D, --debug            Debug mode
182    -E, --error            Error mode
183    -F, --fast             Fast mode
184    -G, --gdb              GDB mode
185    -H, --help            Help mode
186    -I, --include          Include mode
187    -L, --library          Library mode
188    -M, --module           Module mode
189    -N, --no              No mode
190    -O, --output           Output mode
191    -P, --perl            Perl mode
192    -Q, --quiet           Quiet mode
193    -R, --recursive        Recursive mode
194    -S, --switch           Switch mode
195    -T, --test            Test mode
196    -U, --unbuffered        Unbuffered mode
197    -V, --version          Version mode
198    -W, --warning          Warning mode
199    -X, --xargs            Xargs mode
200    -Y, --yargs            Yargs mode
201    -Z, --zargs            Zargs mode
202    -A, --args            Args mode
203    -B, --batch            Batch mode
204    -C, --compact          Compact mode
205    -D, --debug            Debug mode
206    -E, --error            Error mode
207    -F, --fast             Fast mode
208    -G, --gdb              GDB mode
209    -H, --help            Help mode
210    -I, --include          Include mode
211    -L, --library          Library mode
212    -M, --module           Module mode
213    -N, --no              No mode
214    -O, --output           Output mode
215    -P, --perl            Perl mode
216    -Q, --quiet           Quiet mode
217    -R, --recursive        Recursive mode
218    -S, --switch           Switch mode
219    -T, --test            Test mode
220    -U, --unbuffered        Unbuffered mode
221    -V, --version          Version mode
222    -W, --warning          Warning mode
223    -X, --xargs            Xargs mode
224    -Y, --yargs            Yargs mode
225    -Z, --zargs            Zargs mode
226    -A, --args            Args mode
227    -B, --batch            Batch mode
228    -C, --compact          Compact mode
229    -D, --debug            Debug mode
230    -E, --error            Error mode
231    -F, --fast             Fast mode
232    -G, --gdb              GDB mode
233    -H, --help            Help mode
234    -I, --include          Include mode
235    -L, --library          Library mode
236    -M, --module           Module mode
237    -N, --no              No mode
238    -O, --output           Output mode
239    -P, --perl            Perl mode
240    -Q, --quiet           Quiet mode
241    -R, --recursive        Recursive mode
242    -S, --switch           Switch mode
243    -T, --test            Test mode
244    -U, --unbuffered        Unbuffered mode
245    -V, --version          Version mode
246    -W, --warning          Warning mode
247    -X, --xargs            Xargs mode
248    -Y, --yargs            Yargs mode
249    -Z, --zargs            Zargs mode
250    -A, --args            Args mode
251    -B, --batch            Batch mode
252    -C, --compact          Compact mode
253    -D, --debug            Debug mode
254    -E, --error            Error mode
255    -F, --fast             Fast mode
256    -G, --gdb              GDB mode
257    -H, --help            Help mode
258    -I, --include          Include mode
259    -L, --library          Library mode
260    -M, --module           Module mode
261    -N, --no              No mode
262    -O, --output           Output mode
263    -P, --perl            Perl mode
264    -Q, --quiet           Quiet mode
265    -R, --recursive        Recursive mode
266    -S, --switch           Switch mode
267    -T, --test            Test mode
268    -U, --unbuffered        Unbuffered mode
269    -V, --version          Version mode
270    -W, --warning          Warning mode
271    -X, --xargs            Xargs mode
272    -Y, --yargs            Yargs mode
273    -Z, --zargs            Zargs mode
274    -A, --args            Args mode
275    -B, --batch            Batch mode
276    -C, --compact          Compact mode
277    -D, --debug            Debug mode
278    -E, --error            Error mode
279    -F, --fast             Fast mode
280    -G, --gdb              GDB mode
281    -H, --help            Help mode
282    -I, --include          Include mode
283    -L, --library          Library mode
284    -M, --module           Module mode
285    -N, --no              No mode
286    -O, --output           Output mode
287    -P, --perl            Perl mode
288    -Q, --quiet           Quiet mode
289    -R, --recursive        Recursive mode
290    -S, --switch           Switch mode
291    -T, --test            Test mode
292    -U, --unbuffered        Unbuffered mode
293    -V, --version          Version mode
294    -W, --warning          Warning mode
295    -X, --xargs            Xargs mode
296    -Y, --yargs            Yargs mode
297    -Z, --zargs            Zargs mode
298    -A, --args            Args mode
299    -B, --batch            Batch mode
300    -C, --compact          Compact mode
301    -D, --debug            Debug mode
302    -E, --error            Error mode
303    -F, --fast             Fast mode
304    -G, --gdb              GDB mode
305    -H, --help            Help mode
306    -I, --include          Include mode
307    -L, --library          Library mode
308    -M, --module           Module mode
309    -N, --no              No mode
310    -O, --output          
```

shows user.java

```
> Server: [Game] Round 1 started. Use /choice <r/p/s> to make your pick. You have 30s.
/choice r
04/20/2026 23:51:07 [Project.Client.Client] (INFO):
> Processing command: CHOICE
04/20/2026 23:51:07 [Project.Client.Client] (INFO):
> Your choice of rock has been recorded.
04/20/2026 23:51:07 [Project.Client.Client] (INFO):
> Server: [Game] Alice#1 has made their choice.
/users
04/20/2026 23:51:10 [Project.Client.Client] (INFO):
> Processing command: USERS
04/20/2026 23:51:10 [Project.Client.Client] (INFO):
> Known clients:
Alice#1 (you) Ready:[x] Turn:[x] Points:0 Choice:rock
Marisa#2 Ready:[ ] Turn:[ ] Points:0 Choice:[?]
Marisa#3 Ready:[x] Turn:[ ] Points:0 Choice:[?]
04/20/2026 23:51:13 [Project.Client.Client] (INFO):
```

shows after choices are made

```
04/20/2026 23:50:57 [Project.Client.Client] (INFO):
> Server: [Game] Marisa#3 is ready. (2 active)
/users
04/20/2026 23:50:39 [Project.Client.Client] (INFO):
> Processing command: USERS
04/20/2026 23:50:39 [Project.Client.Client] (INFO):
> Known clients:
Alice#1 (you) Ready:[x] Turn:[ ] Points:0 Choice:[?]
Marisa#2 Ready:[ ] Turn:[ ] Points:0 Choice:[?]
Marisa#3 Ready:[x] Turn:[ ] Points:0 Choice:[?]
04/20/2026 23:51:06 [Project.Client.Client] (INFO):
```

shows before choices are made

 Saved: 4/21/2026 12:36:21 AM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain which side is authoritative, what the client mirrors locally, and how the state stays synchronized.

Your Response:

The server is authoritative in this case. All game state lives in the User object stored in BaseServerThread. Client mirrors locally ConcurrentHashMap<Long, User> called knownUsers, all of which mirror the server state, including the ready status, turn status, points, etc. These are updated when the server sends a new payload. They stay synchronized by a one way model, since Server is the sanity check for everything here, it is solely responsible for storing and telling clients what their state ought to be. Essentially, the client only gets its state from what it is told by the server.

 Saved: 4/21/2026 12:36:21 AM

Details:

- Show the full flow for the primary gameplay command for your project.
- Include the key validation gates for this flow (phase, participation, turn/role ownership, and elimination/inactive checks where applicable).
- Primary action by project (your command name may differ slightly):
 - RPS: `/pick` - player locks in their choice for the round
 - Trivia: `/answer` - player locks in their answer for the current question
 - Hangman: `/letter` - player guesses a single letter
 - Battleship: `/attack` - player attacks a coordinate on the opponent's board
 - Drawing: `/guess` - guesser submits a word guess
 - Go Fish: `/target` - player requests a card rank from another player

 Part 1:

Progress: 100%

Details:

- **(code)** Show one contiguous command path: `Client processClientCommand -> ServerThread process* -> Server/GameServer handle* -> GameServer broadcast/unicast/multicast -> ServerThread send* -> Client processPayload`.
- **(runtime)** Include runtime evidence showing one successful execution path and one rejected path caused by phase, turn/role, or elimination/inactive validation (as applicable).
- Clearly caption screenshots by step (`client`, `process`, `handle`, `sync`, `receive`).
- **(runtime)** hints for this task:
 - RPS: show lock-in confirmation and acted-state update
 - Trivia: show lock-in confirmation and round-progress update
 - Hangman: show blanks/strikes update after a letter guess
 - Battleship: show a hit or miss result broadcast
 - Drawing: show role validation and word-match check with round-end path on correct guess
 - Go Fish: show the match-transfer path and the draw-from-deck path

case CHOICE:

```
String choiceText = text.replaceFirst("/choice", "").trim();
sendChoice(choiceText);
return true;
```

Client case choice (CLIENT)

```
switch (incoming.getMessageType()) {
    case CLIENT_CONNECT:
        processClientConnect(incoming);
        break;
    case CLIENT_COMMAND:
        processClientCommand(incoming);
        break;
    case CLIENT_DISCONNECT:
        processClientDisconnect(incoming);
        break;
}
```

Case in serverthread (PROCESS)

Process choice in server thread (PROCESS)

handle choice in game server (HANDLE/SYNC)

Server thread send (SEND)

```
protected boolean sendTurnStatus(long clientId, boolean hasTakenTurn) {
    BoolPayload payload = new BoolPayload();
    payload.setPayloadType(PayloadType.PLAYER_TURN_STATUS);
    payload.setClientId(clientId);
    payload.setValue(hasTakenTurn);
    return sendToClient(payload);
}
```

Server thread send turn status (SEND)

```
private void processTurnStatus(Payload payload) { // You, I think you've changed variable naming, don't start to fail
    string choice = payload.getMessage();
    myUser.setChoice(choice);
    LOGS.PUTL.INSTANCE.info(TEXTX.colorize("Your choice of " + choice + " has been recorded.", COLOR.GREEN));

    private void processTurnStatus(Payload payload) {
        if ((payload instanceof sendPayload)) {
            LOGS.PUTL.INSTANCE.warning("Expected sendPayload for PLAYER_TURN_STATUS. Got: " + payload.getClass());
            return;
        }
        Payload bp = (Payload) payload;
        // user default choice is as a choice trigger
        if (bp.getClientId() == Constants.DEFAULT_CLIENT_ID) {
            // user trigger for all users, update state knownUsers cache
            knownUsers.put(choice, user) > user.setTurnTaken(true);
            LOGS.PUTL.INSTANCE.info(TEXTX.colorize("All users' turn status reset", COLOR.YELLOW));
            return;
        }
        User user = knownUsers.get(bp.getClientId());
        if (user == null) {
            return;
        }
        user.setTurnTaken(bp.getValue());
        // Uncomment for debugging local state synchronization from server turn-status
        payload.setChoice(choice);
        LOGS.PUTL.INSTANCE.info(TEXTX.colorize("Setting forward choice to turnTakenness", user.getDisplayName(), COLOR.PURPLE));
    }
}
```

Client process choice and turn (RECEIVE)

```
04/20/2026 23:51:07 [Project.Client.Client] (INFO):
> Server: [Game] Alice#1 has made their choice.
/choice p
04/20/2026 23:51:13 [Project.Client.Client] (INFO):
> Processing command: CHOICE
04/20/2026 23:51:13 [Project.Client.Client] (INFO):
> Your choice of paper has been recorded.
04/20/2026 23:51:13 [Project.Client.Client] (INFO):
```

shows the choice runtime

```
/choice s
04/21/2026 00:01:54 [Project.Client.Client] (INFO):
> Processing command: CHOICE
04/21/2026 00:01:54 [Project.Client.Client] (INFO):
> Your choice of scissors has been recorded.
04/21/2026 00:01:54 [Project.Client.Client] (INFO):
> Server: [Game] Alice#1 has made their choice.
/choice s
04/21/2026 00:01:57 [Project.Client.Client] (INFO):
> Processing command: CHOICE
04/21/2026 00:01:57 [Project.Client.Client] (WARNING):
> You already made your choice for this round.
```

Rejected path due to erroneous input



Saved: 4/21/2026 12:39:10 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the full flow of this command from client input to server handling to client update.

Your Response:

1. Player types /choice r
2. ProcessCommand -> sendChoice
3. Client updates information

3. Client validates information
4. Passes payload, eg "rock" to server
5. ServerThread receives it in processPayload, matches it to case CHOICE, and calls processChoice, which hands it to server.handlechoice
6. handlechoice runs server side validation, this is authoritative
7. if its approved its sent back down the chain, first it's recorded then a unicast is made which sends the choice back
8. processChoiceConfirmation updates myUser.setChoice(choice) and prints the confirmation



Saved: 4/21/2026 12:39:10 AM

☰ Task #3 (1 pt.) - Secondary or setup action flow

Progress: 100%

Details:

- Show the secondary action or setup flow for your project.
- Include the phase transition or phase restriction that makes this flow valid (or invalid) at the right time.
- Secondary action by project (your command name may differ slightly):
 - RPS: round evaluation and result broadcast - the server-side path that resolves all picks, applies eliminations, and broadcasts scores after the round ends
 - Trivia: round scoring broadcast - the server-side path that scores answers in lock-in order and broadcasts the updated scoreboard
 - Hangman: `/guess` - player guesses the full word directly
 - Battleship: `/place` - player places a ship on their board during the setup phase
 - Drawing: `/draw` - drawer sends coordinate/stroke data to update the shared board
 - Go Fish: pair-removal flow - the server-side path that detects a completed pair or set, removes it from the hand, and broadcasts the update

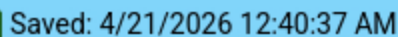
🖼 Part 1:

Progress: 100%

Details:

- **(code)** Show the relevant code path(s) that differ from the primary flow.
- Clearly caption the screenshots with `secondary` plus phase/role context.
- **(runtime)** Include runtime evidence for this flow, including one example that proves phase enforcement or inactive/eliminated-player handling (if applicable).
- **(runtime)** hints for this task:
 - RPS: show the resolution path through all picks, the elimination check, and the scoreboard broadcast
 - Trivia: show the post-round scoring path with diminishing points and scoreboard broadcast
 - Hangman: show a correct full-word guess path and an incorrect full-word guess path
 - Battleship: show ship placement validation and the transition out of the setup phase
 - Drawing: show drawer role validation, a coordinate broadcast to guessers, and a

- Go Fish: show pair detection, removal from hand, and hand-count broadcast



1

- 1

—

compares all choices and actually sets the phase and so forth, and it's what is responsible for actually driving the game



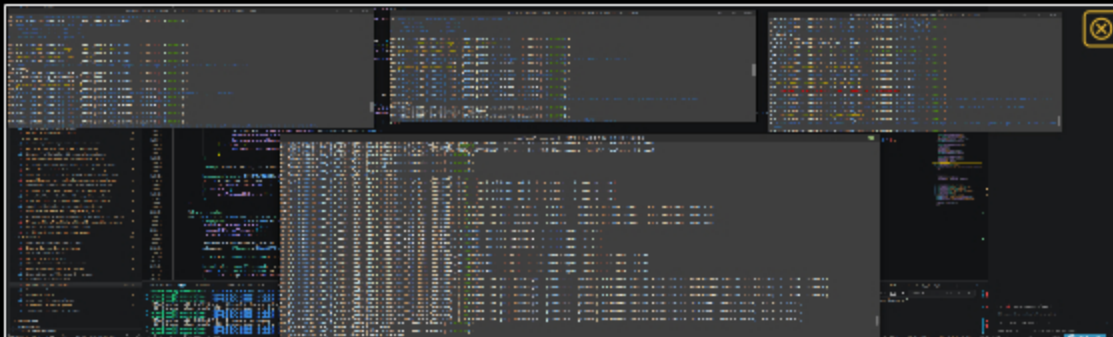
Saved: 4/21/2026 12:40:37 AM

Task #4 (1 pt.) - Game Cycle Demo

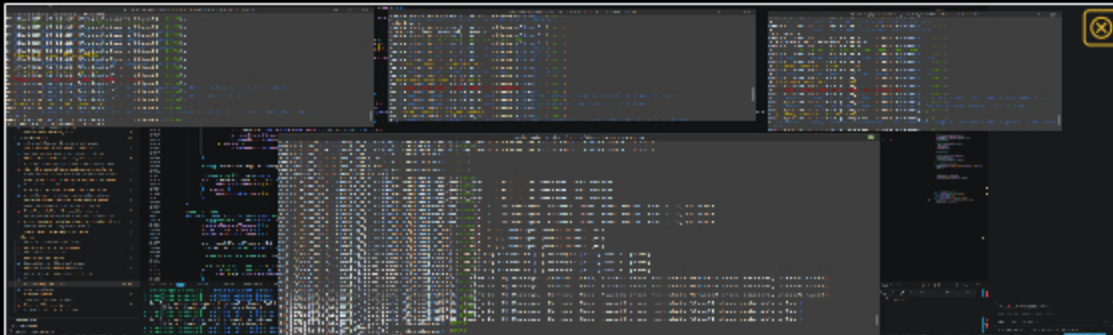
Progress: 100%

Details:

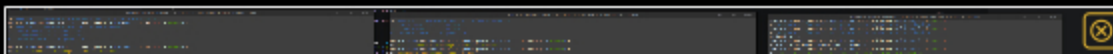
- Show terminal examples of a full session demonstrating the project's main commands and progress output.
- Ensure at least 3 clients and the server are shown.
- Clearly caption screenshots.
- Make sure the screenshots show at least these milestones:
 - session start or ready-check completion
 - one clear phase transition relevant to your project (for example setup->action or round start->round end)
 - at least one successful gameplay action
 - at least one rejected action due to phase/turn/role/elimination rules (if applicable)
 - score, points, elimination, or round-result output as applicable
 - session end or clear next-round progression

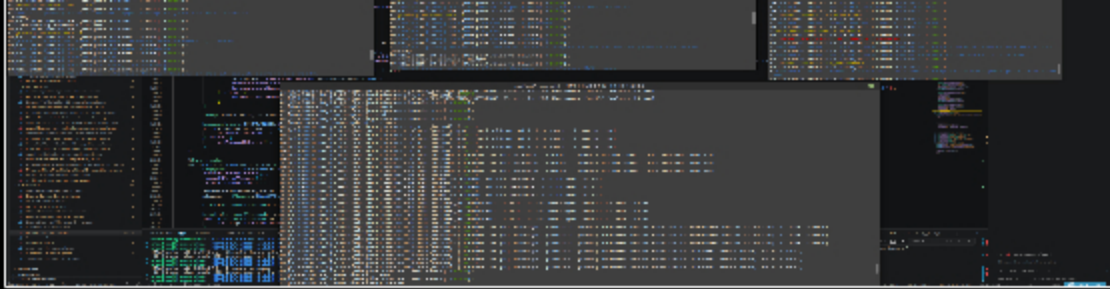
A screenshot of a terminal window displaying the output of a game server. The text shows a 'READY CHECK' phase followed by a 'START' phase, indicating the beginning of a round. The output is color-coded with green for success and red for errors or warnings.

Shows the start/ready check section of a round

A screenshot of a terminal window displaying the output of a game server. The text shows a 'ROUND FINISH' phase followed by a 'TIE' message, indicating the end of a round. The output is color-coded with green for success and red for errors or warnings.

Round finishes with a tie

A screenshot of a terminal window displaying the output of a game server. The text shows a 'READY CHECK' phase followed by a 'START' phase, indicating the beginning of a round. The output is color-coded with green for success and red for errors or warnings.



Shows result of two rounds

```
/choice s
04/21/2026 00:01:54 [Project.Client.Client] (INFO):
> Processing command: CHOICE
04/21/2026 00:01:54 [Project.Client.Client] (INFO):
> Your choice of scissors has been recorded.
04/21/2026 00:01:54 [Project.Client.Client] (INFO):
> Server: [Game] Alice#1 has made their choice.
/choice s
04/21/2026 00:01:57 [Project.Client.Client] (INFO):
> Processing command: CHOICE
04/21/2026 00:01:57 [Project.Client.Client] (WARNING):
> You already made your choice for this round.
```

Pulled from example but shows how entering something twice while a round is in progress fails

 Saved: 4/21/2026 12:13:47 AM

Section #4: (1 pt.) Misc

Progress: 95%

Task #1 (0.33 pts.) - Github Details

Progress: 87%

Part 1:

Progress: 75%

Details:

- From the Commits tab of the Pull Request screenshot the commit history.



Missing Caption

Part 2:

Progress: 100%

Details:

- Include the link to the Pull Request (should end in `/pull/#`).

URL #1

<https://github.com/methodius54/si329-it114-004-2026/pull/7>



URL

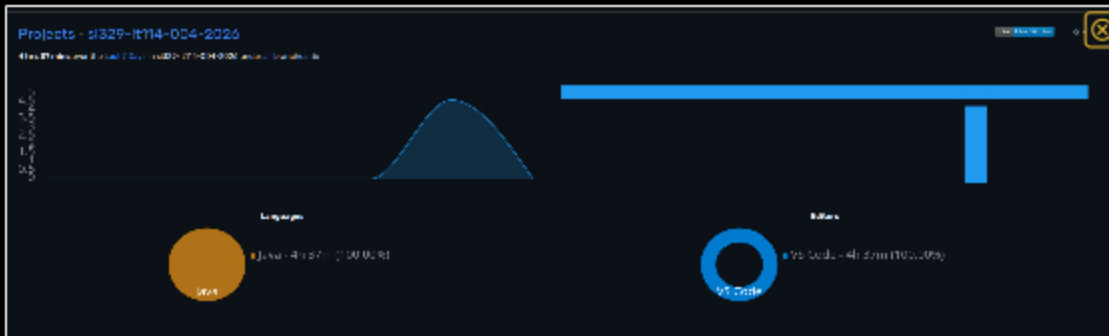
<https://github.com/methodius54/>

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard.
- Click **Projects** and find your repository.
- Capture the overall time at the top that includes the repository name.
- Capture the individual time at the bottom that includes the file time.
- The duration is not relevant for the grade and the visual graphs are not necessary.



Shows wakatime dashboard

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question in at least a few decent sentences.

Your Response:

I learned a good chunk on how networking works, specifically how information is passed along and how information is synchronized. It helped me understand how specifically threads and servers work, and I understand that it is a sort of process up, with the serverthread handling the individual case of one client instance



Saved: 4/21/2026 12:18:32 AM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question in at least a few decent sentences.

Your Response:

The easiest part was probably just setting everything up. It was pretty easy to simply copy some of the existing methods to pass input along.



Saved: 4/21/2026 12:17:20 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question in at least a few decent sentences.

Your Response:

The hardest part of this assignment was figuring out evaluation as well as struggling with an error I had made earlier in the code in User.java (one letter typo btw), which was causing an issue where due to the fact that it was receiving the call to end the session, it would keep running the round endlessly irrespective of the fact that the users had made their choices



Saved: 4/21/2026 12:17:19 AM